DATA SCIENCE • MODULE 01

Python for Data Science

Your Programming Foundation for Every DS Task

■ PYTHON PROGRAMMING

■ LEARNING OUTCOME

By the end of this PDF you will write Python confidently for data science: using NumPy arrays, Pandas DataFrames, file I/O, and Pythonic coding patterns used by professional data scientists.

■ Skills You Will Gain
• Set up Python environment (Anaconda / Colab)
• Use variables, loops, functions, and list comprehensions
• Master NumPy arrays and vectorized operations
• Use Pandas for data loading, filtering, and aggregation
• Read and write CSV, Excel, and JSON files
• Write clean, reusable Python functions for DS pipelines

■ **Python is the #1 language for Data Science. 90% of DS job postings require Python. Learning it once unlocks ML, AI, analytics, and automation.**

SECTION 1

Python DS Environment Setup

1	**Install Anaconda** Download from anaconda.com — includes Python, Jupyter, NumPy, Pandas in one click
2	**Or Use Google Colab** colab.research.google.com — free browser-based Jupyter with free GPU, no install needed
3	**Create Environment** conda create -n dsenv python=3.11 → conda activate dsenv
4	**Install Libraries** pip install numpy pandas matplotlib seaborn scikit-learn
5	**Launch Jupyter** jupyter notebook → opens in browser at localhost:8888

SECTION 2

Python Basics for Data Science

```
# Data types x = 42 # int price = 19.99 # float name = 'Alice' # str flag = True # bool # Lists and dictionaries scores = [88, 92, 76, 95, 83] student = {'name':'Alice', 'age':22, 'gpa':3.8} # List comprehension (Pythonic & fast) squares = [x**2 for x in range(1,11)] even = [x for x in scores if x >= 90] # Function def clean_text(text): return text.strip().lower().replace(' ','_') # Lambda (one-line function) double = lambda x: x * 2
```

SECTION 3

NumPy — The Engine of DS

```
import numpy as np # Create arrays arr = np.array([1,2,3,4,5]) matrix = np.zeros((3,3)) # 3x3 zeros rnd = np.random.randn(100) # 100 random normal values # Vectorized math (NO loops needed!) arr * 2 # [2,4,6,8,10] arr ** 2 # [1,4,9,16,25] np.sqrt(arr) # element-wise square root # Stats np.mean(arr), np.std(arr), np.median(arr) np.min(arr), np.max(arr), np.sum(arr) # Slicing arr[2:5] # index 2,3,4 matrix[0,:] # first row matrix[:,1] # second column # Boolean indexing arr[arr > 3] # array([4,5])
```

SECTION 4

Pandas — The Heart of Data Science

```
import pandas as pd # Load data df = pd.read_csv('data.csv') df = pd.read_excel('data.xlsx', sheet_name='Sheet1') df = pd.read_json('data.json') # Inspect df.head(5) # first 5 rows df.info() # types + nulls df.describe() # stats summary df.shape # (rows, cols) # Select df['Age'] # single
```

```
column (Series) df[['Name','Age']] # multiple columns (DataFrame) df.iloc[0:5, 1:3] # rows 0-4,
cols 1-2 by position df.loc[df['Age']>25] # rows where Age > 25 # Add / modify column df['BMI'] =
df['Weight'] / (df['Height']**2) # Group & aggregate df.groupby('Dept')['Salary'].mean()
df.groupby('City').agg({'Sales':'sum', 'Orders':'count'}) # Sort df.sort_values('Salary',
ascending=False).head(10)
```

SECTION 5

File I/O and Data Pipelines

```
# Save cleaned data df.to_csv('clean_data.csv', index=False) df.to_excel('report.xlsx',
sheet_name='Results', index=False) # Merge two DataFrames (like SQL JOIN) result =
pd.merge(orders_df, customers_df, on='customer_id', how='left') # Concatenate DataFrames (stack
rows) combined = pd.concat([df_jan, df_feb, df_mar], ignore_index=True) # Apply a function to
every row df['Grade'] = df['Score'].apply(lambda x: 'Pass' if x>=50 else 'Fail') # Pivot table
pivot = df.pivot_table(values='Revenue', index='Month', columns='Region', aggfunc='sum',
fill_value=0)
```

SECTION 6

Python for DS Workflow

1	**Load** pd.read_csv() / read_excel() / read_json()
2	**Inspect** head(), info(), describe(), shape, dtypes
3	**Clean** isnull(), fillna(), dropna(), drop_duplicates(), astype()
4	**Transform** apply(), map(), groupby(), merge(), pivot_table()
5	**Analyze** NumPy stats, correlation, value_counts()
6	**Visualize** matplotlib / seaborn (next module)
7	**Export** to_csv(), to_excel() for sharing results

■ PRACTICE QUESTIONS

Q1. What is the difference between a Python list and a NumPy array?

Answer: *NumPy arrays are faster (vectorized C operations), support element-wise math, and are memory-efficient. Lists require loops for math operations.*

Q2. Write a one-line list comprehension to get squares of even numbers from 1 to 20.

Answer: *[x**2 for x in range(1,21) if x%2==0]*

Q3. How do you select all rows where Salary > 50000 in a Pandas DataFrame?

__Answer:__ df[df['Salary'] > 50000] or df.loc[df['Salary'] > 50000]

Q4. What does df.groupby('Region')['Revenue'].sum() return?

__Answer:__ Total revenue for each unique Region — a Series indexed by Region name.

Q5. What is the difference between iloc and loc in Pandas?

__Answer:__ iloc uses integer positions (0,1,2...); loc uses label/index names or boolean conditions.

■ MINI PROJECT

Project: Student Performance Analysis
Apply everything in this module with a real dataset!

Dataset: student_scores.csv (Name, Subject, Score, Attendance, Grade)

Steps:

1	**Load** pd.read_csv() → df.head(), df.info()
2	**Filter** Students with Score > 80 AND Attendance > 75%
3	**Aggregate** Average score per Subject using groupby
4	**New Column** Create 'Status' = Pass if Score>=50 else Fail
5	**Export** Save cleaned+enriched data to new CSV

Expected Output: Cleaned DataFrame + summary stats + exported CSV

Key Insights to Find: Top subject, pass/fail rate, correlation between attendance and score

■ BONUS RESOURCES

Python Docs	docs.python.org/3 — official reference
NumPy Docs	numpy.org/doc/stable — array operations
Pandas Docs	pandas.pydata.org — complete API reference
Practice	leetcode.com (Easy Python), HackerRank Python track
Book	'Python for Data Analysis' by Wes McKinney (Pandas creator)
Free Course	CS50P — Harvard Python course (free on edX)

■ Python for Data Science Cheat Sheet	
`import numpy as np`	`Import NumPy`
`import pandas as pd`	`Import Pandas`
`np.array([1,2,3])`	`Create NumPy array`

`np.zeros((r,c))`	Matrix of zeros
`np.mean/std/sum(arr)`	Array statistics
`arr[arr > x]`	Boolean indexing
`pd.read_csv('f.csv')`	Load CSV
`df.head() / df.info()`	Inspect DataFrame
`df['col']`	Select column
`df.loc[condition]`	Filter rows
`df.groupby('col').agg()`	Group & aggregate
`df.merge(df2, on='key')`	Join DataFrames
`df['col'].apply(func)`	Apply function
`df.to_csv('out.csv')`	Export to CSV